

RAG Pipeline Step 5: Routing

1. Definition

Routing is a "pre-retrieval" step that acts as a "traffic cop" for a user's query. Its job is to decide *where* to send the query when you have multiple potential destinations (e.g., multiple vector stores, a SQL database, or a simple chatbot).

This is a critical technique for building complex, multi-source agents that can handle diverse tasks. For example, a query about "last quarter's sales" should be routed to a SQL database, while a query about "our company's leave policy" should be routed to an HR vector store.

There are two primary methods for this, which you correctly identified:

2. Logical Routing

- **What is it?** This method uses the **reasoning (logic) of an LLM** to decide which route to take. It is powerful and flexible but slower and more expensive, as it requires an LLM call.
- **When to use it:** Use this when the decision-making is complex and requires understanding nuance, or when you need to route to a specific "tool" (like a SQL chain).
- **How it works (Process):**
 1. The user's query is fed into a prompt.
 2. This prompt also contains *descriptions* of the available routes. For example:
 - Route 1 (Name: 'hr_policy'): "Good for questions about employee benefits, time off, and company HR policies."
 - Route 2 (Name: 'product_docs'): "Good for technical questions about product features, APIs, and how-to guides."
 3. An LLM (often one that supports **Tool Calling** or **Structured Output**) reads the query and the descriptions, and its job is to output the *name* of the best route (e.g., `{'destination': 'hr_policy'}`).
- **Key LangChain Tool:**
 - `RunnableBranch` : This is the modern LCEL (LangChain Expression Language) component for routing. You create a "branch" that first calls an LLM chain to get the route name, and then provides a (route_name, chain) mapping to direct the flow.
 - `MultiRetrievalQAChain` (**Legacy**): This is a pre-built chain that bundles this logic. It uses an `LLMRouterChain` internally to choose from a list of retrievers based on their descriptions.

3. Semantic Routing

- **What is it?** This method uses **embedding similarity** to decide which route to take. It is extremely fast and cheap because it does not require an LLM call at runtime.
- **When to use it:** Use this when you have clearly distinct categories of queries and want a very fast, low-latency "first-pass" filter.
- **How it works (Process):**
 1. **Offline (Indexing):** You define your routes and provide a few example phrases (called "utterances") for each.
 - Route: 'weather'
 - Utterances: ["what's the forecast", "is it going to rain", "temperature today"]
 - Route: 'finance'
 - Utterances: ["stock price", "quarterly earnings", "company revenue"]
 2. You then *embed* these utterances and store them.
 3. **Online (Runtime):** When a user's query comes in ("is it going to be sunny tomorrow?"), it is embedded.
 4. The query's vector is compared (using cosine similarity) to the vectors of all the *example utterances*.
 5. The query is instantly routed to the chain whose utterances are most semantically similar.
- **Key LangChain Tool:**
 - `SemanticRouter` : This is a popular, separate library (`semantic-router`) that integrates seamlessly with LangChain. You define `Route` objects with their example utterances, and it creates a router "layer" that you can place at the beginning of your chain to direct traffic based on similarity scores.